FACULDADE VINCIT

Organização e Manipulação de Estrutura de Dados

5. Cenários para Aplicação dos Algoritmos de Ordenação Complexos

Ineficiência dos algoritmos $O(n^2)$

FACULDADE VINCIT

Como já comentado em aulas anteriores, os algoritmos de ordenação mais simples, como Bubble Sort, Selection Sort e também Insertion Sort são ineficientes em ordenações de grande volume de elementos.

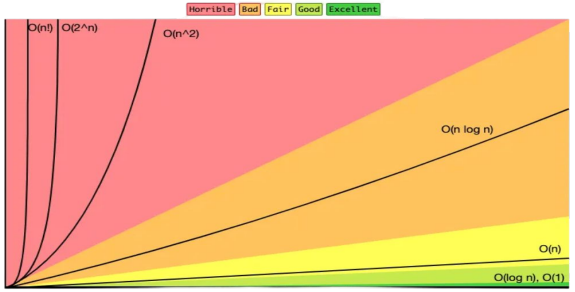
Ainda que possuam complexidade cognitiva muito baixa, sendo muito fácil de serem entendidos e também de serem implementados, esses algoritmos requerem muito poder computacional para ordenar grandes arranjos, sempre tendendo à n^2 em número de comparações no pior caso.

Além disso, esses algoritmos possuem execução linear, necessitando de uma ordem sequencial de comparações e trocas, impedindo que possam ser modificados para funcionar em cenários com recursividade e paralelismo.

Algoritmos de Ordenação Complexos

FACULDADE VINCIT

Em grandes coleções de elementos, recomenda-se algoritmos de ordenação mais complexos, que utilizam outras estratégias para ordenação do conjunto. Na maioria dos casos, essas estratégias não são tão intuitivas, e são comumente pensadas para que o computador possa ordenar os elementos de acordo com sua capacidade.



Todavia a complexidade cognitiva do algoritmo aumenta significativamente, necessitando de uma atenção acurada do programador no momento de sua implementação, o que torna a compreensão do algoritmo muito mais dificultosa.

A grande vantagem dos algoritmos de ordenação complexos, em relação aos de complexidade cognitiva baixa, é a sua velocidade. As estratégias utilizadas por tais algoritmos otimizam o trabalho do computador, avaliando outros aspectos/propriedades dos elementos para realizar sua ordenação.

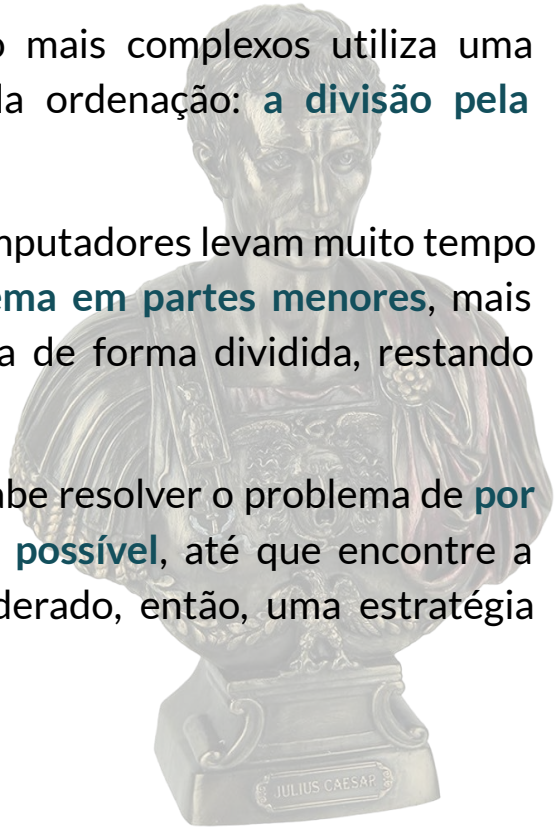
Estratégia Dividir para Conquistar

FACULDADE VINCIT

Uma parte considerável dos algoritmos de ordenação mais complexos utiliza uma estratégia interessante para solucionar o problema da ordenação: **a divisão pela conquista**.

Diante de um grande problema, tanto humanos como computadores levam muito tempo para encontrar uma solução, mas, ao **quebrar o problema em partes menores**, mais simples de se entender, a solução pode ser encontrada de forma dividida, restando apenas juntar a solução do problema.

Nessa estratégia, pressupomos que o computador não sabe resolver o problema de **por inteiro**, sendo necessário a **divisão até a menor parte possível**, até que encontre a menor parte solucionável pelo algoritmo, sendo considerado, então, uma estratégia recursiva.



Principais Algoritmos de Ordenação

FACULDADE VINCIT

São inúmeros algoritmos de ordenação disponíveis para se trabalhar com grandes coleções de dados. Os mais citados e também utilizados costumam ser Quick Sort, Merge Sort, Heap Sort e Radix Sort, sempre variando em relação à sua aplicação e também complexidade assintótica.

Algoritmos de Ordenação Complexos			
Algoritmo	**Pior Caso**	**Melhor Caso**	**Caso Médio**
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Radix Sort	$O\ (n \times k)$	$O\ (n \times k)$	$O\ (n \times k)$

Algoritmos Complexos - Aplicações

FACULDADE VINCIT

Os algoritmos de ordenação complexos têm uma ampla gama de aplicações em diversos campos da ciência da computação e da engenharia de software. Aqui estão algumas das principais aplicações para cada um dos algoritmos de ordenação complexos mencionados:

- Sistemas de Bancos de Dados relacionais;
- Sistemas Operacionais;
- Processamento de Textos;
- Ordenação de Listas Grandes;
- Sistemas de Arquivos e mídias digitais;
- Sistema de Tempo Real;
- Prioridade de Fila;
- Ordenação de Strings;
- Dados Científicos;
- Aplicações Financeiras;
- Sistemas Eleitorais.



Algoritmos Complexos - Eficiência

FACULDADE VINCIT

A eficiência é uma característica que varia de algoritmo para algoritmo. Não somente é considerada a velocidade de ordenação, mas também o consumo de recursos de cada algoritmo, além da análise assintótica de cada cenário aplicado.

Muitos algoritmos têm destaque em relação à sua velocidade, como *Merge Sort* e o *Radix Sort*, porém seu consumo de memória pode ser considerado elevado, e até em alguns casos, exacerbado para determinado cenário.

Por isso, a escolha do algoritmo de forma adequado, dimensionado ao cenário em questão, pode uma grande eficiência, levando-se em consideração capacidade de processamento e alocação de memória.

FACULDADE VINCIT

fb.com/faculdade.vincit

linkedin.com/school/faculdade_vincit

instagram.com/faculdade_vincit